Data Pipelines

ingest, transform, orchestrate, test

Watch the 4-min video

In one line: a data pipeline **ingests** data from apps, databases and SaaS tools, lands it **raw**, **transforms** it with SQL through bronze → silver → gold layers in a lakehouse or warehouse, is **orchestrated** on a schedule, and is guarded by **data-quality tests** so dashboards only ever read trustworthy tables. ELT is the modern default.

1 One simple question

The CEO asks: *how many users who signed up last week made a purchase?* Sign-ups are app events, purchases are in the payments database, and campaign data sits in a CRM. Answering once is a few hours of exports; answering correctly every morning is a pipeline. It has four parts: ingestion, transformation, orchestration and quality.

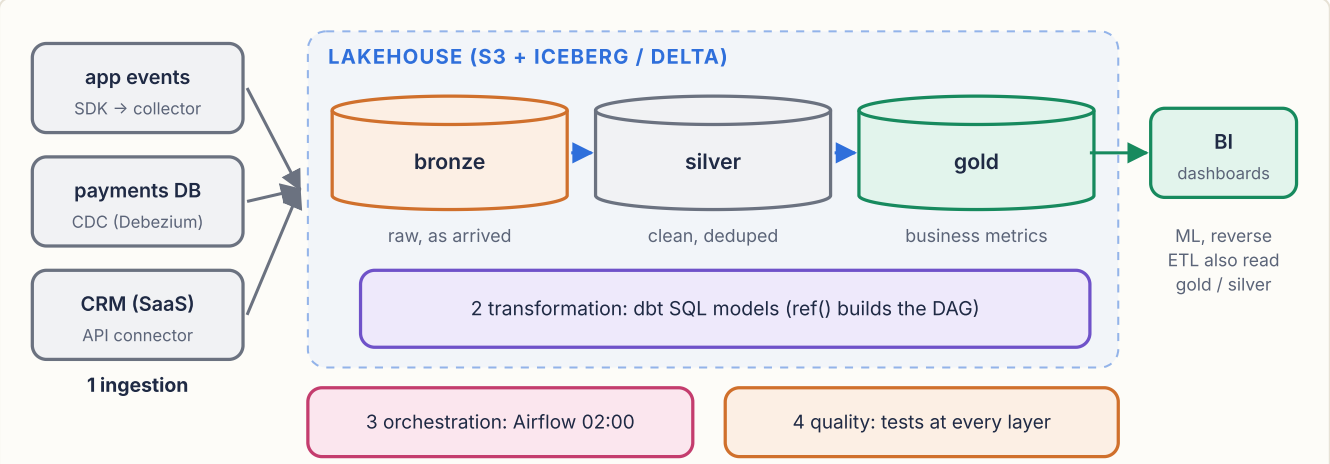


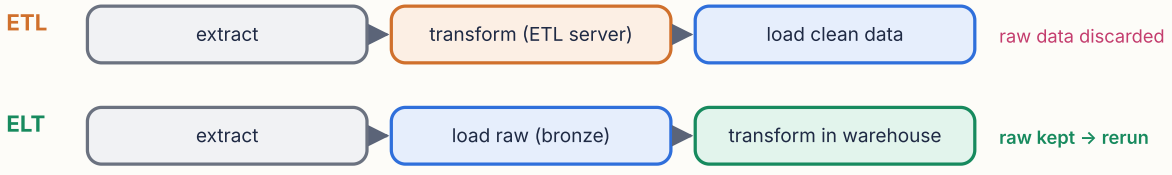
Figure 1. A typical modern pipeline. Raw data is kept, transformations are SQL, and every layer is tested before the next is built.

2 Ingestion: getting data in

Source	How	Watch out for
App and web events	SDK → collector endpoint → Kafka or straight to object storage as files	duplicates from retries (keep an event ID), clock skew, schema changes
Operational databases	**Change data capture**: Debezium reads the MySQL binlog or Postgres WAL (logical replication) and emits every insert/update/delete	full snapshot first, then the stream; ordering per key
SaaS tools (CRM, ads)	API connectors (Fivetran, Airbyte, custom), incremental by cursor	rate limits, API changes, late updates
Files from partners	SFTP / bucket drops, picked up by a sensor	partial files: wait for a success marker

Prefer CDC to nightly `SELECT * WHERE updated_at > …` extracts: the query approach misses hard deletes and rows updated without touching `updated_at`, and loads the production database.

3 ETL or ELT



ETL still fits when data must be masked or filtered before it may land (e.g. PII rules), or the target is small.

Figure 2. The difference is where the transform runs, and whether the raw data survives.

In **ETL** data is cleaned on an ETL server before loading, so only the transformed result exists; a mistake in the transform means re-extracting from sources that may have changed. In **ELT** raw data is loaded first and transformed inside a scalable warehouse or lakehouse with SQL. Cheap object storage and powerful engines (Snowflake, BigQuery, Databricks, Trino) made ELT the default: raw data is kept, so any transformation can be changed and rerun, exactly like batch reruns in lesson 31.

4 The lakehouse

A **data lake** is files (Parquet) in object storage: cheap and open, but plain files have no transactions, so a failed job leaves half-written data and concurrent writers clash. **Table formats** such as Delta Lake, Apache Iceberg and Apache Hudi add a metadata layer (Delta's transaction log, Iceberg's snapshots and manifests) on top of the same files, giving:

- **ACID commits:** a write becomes visible atomically, or not at all; `MERGE INTO` for upserts from CDC.
- **Schema evolution:** add or rename columns without rewriting data, and reject writes that don't match.
- **Time travel:** query the table as of an earlier snapshot, to debug or to roll back a bad load.
- **Many engines, one copy:** Spark, Trino, Flink and warehouses can read the same tables.

5 Medallion layers and dbt

- **Bronze:** raw, exactly as it arrived, append-only, partitioned by load date. Never edited; it is your replay source.
- **Silver:** cleaned and conformed: types cast, duplicates removed by event ID, CDC applied to give current rows, test accounts dropped, entities joined (users, orders, sessions).
- **Gold:** business-ready tables and metrics (daily active users, revenue by country, weekly conversion) that dashboards read directly, with agreed definitions.

dbt turns each transformation into a SQL `SELECT` model; `ref()` declares dependencies so dbt builds them in order, as tables, views or incremental merges, and runs tests after each build.

```
-- models/gold/signup_purchase_weekly.sql
select date_trunc('week', u.signed_up_at) as week,
       count(distinct u.user_id)           as signups,
       count(distinct o.user_id)           as buyers
from {{ ref('silver_users') }} u
left join {{ ref('silver_orders') }} o
  on o.user_id = u.user_id and o.created_at < u.signed_up_at + interval '7' day
group by 1

# models/silver/schema.yml  (tests run after every build)
- name: silver_orders
  columns:
    - name: order_id    tests: [unique, not_null]
    - name: user_id     tests: [not_null, relationships: {to: ref('silver_users'), field: user_id}]
```

Figure 3. A gold model answering the CEO's question, and the silver tests that must pass before it is built.

6 Data quality: stop bad data before gold

Pipelines rarely fail loudly; they produce plausible wrong numbers. Encode expectations as tests at every layer:

- **Schema and constraints:** `not_null`, `unique`, `accepted_values`, `relationships` (dbt's built-in generic tests), or Great Expectations suites.
- **Volume and distribution:** today's row count within the usual range; revenue not 10× yesterday's.
- **Freshness:** the source's newest record is less than N hours old (`dbt source freshness`).

A failing test **stops the downstream models** and alerts the owner: an empty dashboard tile is better than a wrong number in a board meeting. Lineage (which tables feed which) shows what a failure affects.

WORKED EXAMPLE: EVENTS → S3 → DBT → WAREHOUSE

The app sends `signup` and `purchase` events to a collector, which writes them as hourly Parquet files to `s3://lake/bronze/events/dt=…/`; Debezium streams the payments database into `bronze.payments`. At 02:00 Airflow loads new bronze partitions, then runs `dbt build`: silver models dedupe events by `event_id`, merge CDC rows into current orders, and drop internal test users; tests check uniqueness, null IDs and that every order has a user. Only if they pass are gold tables like `signup_purchase_weekly` rebuilt. The BI tool reads gold; the CEO's question is one row: 41,200 sign-ups, 6,950 buyers within 7 days.

TRADE-OFFS

- ELT keeps raw data and flexibility but stores more and can expose raw PII unless access is controlled.
- Daily batch is simple and cheap; streaming ingestion (lesson 32) gives fresher gold tables at more operational cost.
- Strict tests that block the pipeline protect trust but can delay all dashboards for one bad source.
- Managed connectors save engineering time but cost money and limit control.

COMMON MISTAKES

- Dashboards reading bronze or silver tables directly, each with its own metric definition.
- Incremental extracts by `updated_at` that silently miss deletes.
- No dedup by event ID, so retried events inflate counts.
- Tests that only alert while the bad data still flows into gold.

7 Key terms

ELT: load raw first, transform inside the warehouse

Lakehouse: object-storage files + table format with ACID

dbt model: a SELECT statement materialised as a table/view

CDC: streaming row changes from a database's log

Medallion: bronze (raw), silver (clean), gold (business)

Data test: assertion on data that blocks bad loads

8 Interview questions

Q1. Design a pipeline from app events to a BI dashboard. BASIC

Land raw events in object storage, load into a lakehouse or warehouse, transform with SQL through bronze, silver and gold, orchestrate with a scheduler, test quality at every step, and point the dashboard at gold tables.

Q2. ETL or ELT? BASIC

ELT is the modern default: load raw data first, so transformations can be changed and rerun; ETL when data must be filtered or masked before it lands.

Q3. Revenue on the dashboard doubled overnight. How would your pipeline have caught it? SCENARIO

Volume and distribution tests on silver and gold (e.g. revenue within a band of the trailing average) would fail and stop the gold build, alerting the owner.

Q4. How do you ingest a production Postgres database without loading it? SCENARIO

Use CDC from the write-ahead log (logical replication, e.g. Debezium): one initial snapshot, then a stream of changes, merged into silver tables.

Q5. What does a table format like Iceberg add over Parquet files? TRICKY

A metadata layer with atomic snapshots: ACID commits, schema evolution, time travel and safe concurrent writers, readable by many engines.

Q6. Why keep bronze if silver is clean? TRICKY

Bronze is the replay source: when a silver rule is wrong, you fix it and rebuild from bronze; without it you'd have to re-extract from sources that have changed.

Next: SD-Q-010 · Time-Series Data backendroadmap.com